THE HIDDEN THIEF

DOCUMENTO TÉCNICO DE DISEÑO · VERSIÓN 1.0

Especificación consolidada del videojuego terminado. Integra las decisiones de diseño y la implementación real en una única estructura, sin separar borrador y versión final.

Motor: Godot 4.6.1 | Lenguaje: GDScript | Plataforma: Windows x86_64 | Género: sigilo 2D top-down

Alcance y criterio documental

El presente TDD describe la versión 1.0, confirmada como completa y jugable por su desarrollador. Se utiliza el comportamiento de los scripts revisados como referencia para especificar la implementación; los elementos de diseño no corroborados por código se identifican expresamente como criterios visuales o de diseño. Los futuros parches no forman parte de los requisitos de esta versión.

1. Concepto, objetivos y estructura de la campaña

El jugador controla a un ladrón que se infiltra en edificios durante un apagón. El desafío consiste en conseguir accesos, sustraer el tesoro y escapar bajo presión temporal. La campaña utiliza tres escenarios: Museo (estatuilla antigua), Mansión (joya valiosa) y Banco (diamante).

Videojuego 2D con perspectiva top-down desarrollado en Godot. El jugador controla a un ladrón que aprovecha un apagón para infiltrarse en distintos lugares protegidos. El escenario permanece oscuro durante toda la partida. Los guardias patrullan utilizando linternas, cuyo haz representa de forma visible su campo de visión.

El objetivo consiste en explorar el escenario, obtener llaves, abrir accesos restringidos y alcanzar el tesoro principal sin ser atrapado. Algunas llaves se encuentran en habitaciones y otras son transportadas por guardias. Tras robar el tesoro se activa una alarma y comienza una cuenta regresiva: el jugador debe alcanzar la salida antes de la llegada de la policía.

Título: The Hidden Thief. Género: sigilo 2D con perspectiva cenital (top-down), movimiento en ocho direcciones y estética pixel art. Motor de desarrollo: Godot 4.6.1. Plataforma de distribución comprobada: Windows x86_64. La campaña comprende tres niveles: Museo, Mansión y Banco, con un tutorial independiente.

Objetivo jugable: infiltrarse, evitar la detección, obtener llaves, acceder al tesoro principal y regresar al punto de salida antes de que finalice el tiempo de escape. El robo activa una alarma y transforma la fase de infiltración en una huida contrarreloj.

Estado del producto: versión 1.0 terminada según el desarrollador. Las mejoras y correcciones posteriores se registrarán como mantenimiento, sin considerarlas requisitos pendientes de esta versión.

2. Plataforma, herramientas y recursos de producción

La versión implementada utiliza Godot 4.6.1 y GDScript; el ejecutable comprobado corresponde a Windows x86_64. Se emplea pixel art de base 32 × 32 píxeles y herramientas gráficas como LibreSprite. El versionado mediante Git figura en el planteamiento del proyecto.

3. Arquitectura general y gestión global

GameManager.gd (Autoload) concentra nivel_actual, lista de rutas de los tres escenarios, posesión del tesoro, alarma, temporizador, estados de finalización y señales globales. Sus operaciones incluyen cargar_nivel(), reiniciar_nivel_actual(), registrar_robo_tesoro(), activar_alarma(), restaurar_alarma() y completar_nivel().

SaveManager.gd (Autoload) centraliza preferencias, dificultad, progresión, checkpoint y persistencia JSON. AudioManager.gd (Autoload) administra música y efectos sonoros a través de buses de audio.

La arquitectura distribuye responsabilidades: nivel.gd recupera jugador y guardias; las puertas y las llaves restauran su propio estado desde SaveManager; tesoro.gd se elimina al cargar si el robo ya está registrado. La interfaz reacciona a señales de GameManager y del jugador.

Escenas relevantes: menu_principal.tscn, tutorial.tscn, presentacion_nivel.tscn, nivel_museo.tscn, nivel_mansion.tscn, nivel_banco.tscn y final_juego.tscn.

4. Jugador: movimiento, animaciones y controles

El protagonista utiliza CharacterBody2D y move_and_slide() para el desplazamiento. La dirección se normaliza para evitar mayor velocidad en diagonal. Dispone de animaciones de caminata en ocho direcciones y estado Idle.

Parámetros de referencia en player.gd: velocidad normal 150 px/s; sigilo 70 px/s; carrera 230 px/s. El sigilo tiene prioridad si se presionan simultáneamente las acciones de sigilo y carrera. Las animaciones modifican su speed_scale según el modo: aproximadamente 0,6 en sigilo, 1 en marcha y 1,5 en carrera.

Entradas implementadas: desplazamiento mediante acciones de movimiento; interacción contextual; stealth; run; y pause, esta última procesada por el HUD. La interacción usa AreaInteraccion para detectar cuerpos y áreas superpuestos, e invoca métodos disponibles en el objeto, como quitar_llave(), robar() e intentar_abrir().

El ladrón no utilizará una animación específica de agacharse. Los tres modos emplearán las animaciones de caminar, modificando velocidad de desplazamiento, velocidad de reproducción y ruido generado.

Controles definidos:

WASD o flechas: movimiento en ocho direcciones.

E: interacción contextual (recoger/robar llave, abrir puerta, tomar tesoro).

Barra espaciadora: modo sigilo mientras se mantiene presionada.

Shift: correr mientras se mantiene presionada.

P: pausa.

5. Sigilo, ruido y percepción auditiva

El ruido está vinculado a la forma de desplazamiento: permanecer inmóvil o moverse en sigilo evita la detección auditiva, mientras caminar y correr aumentan el radio efectivo. La detección visual continúa activa durante el sigilo.

player.gd expone obtener_radio_ruido(). El jugador quieto o desplazándose en sigilo tiene radio de ruido 0; caminando normalmente, 120 unidades; corriendo, 220 unidades. Esta magnitud se utiliza en el sistema auditivo de los guardias.

El sonido de pasos se gestiona con AudioStreamPlayer2D, ajustando volumen y pitch según la forma de desplazamiento. La ausencia de ruido detectable en sigilo no equivale a invisibilidad: el jugador puede ser descubierto por el cono visual de una linterna.

El guardia aplica un multiplicador de audición durante la alarma en la lógica de patrulla. La respuesta al ruido en las fases de alerta sigue su implementación específica, por lo que no se afirma que todas las fases compartan idéntico multiplicador.

6. Guardias: patrulla, alerta, investigación y persecución

La investigación no es un cuarto valor del enum principal: se implementa como parte del estado ALERTA, dividido en fases YENDO y ESPERANDO. La navegación se resuelve mediante NavigationAgent2D y objetivos de patrulla.

guardia.gd implementa una máquina de estados con PATRULLA, ALERTA y PERSECUCION. La alerta se subdivide en las fases YENDO y ESPERANDO. El desplazamiento utiliza NavigationAgent2D y puntos de patrulla configurables mediante Marker2D.

En patrulla, el guardia se desplaza entre objetivos; ante un ruido o indicio de presencia, pasa a investigar. En persecución, se dirige hacia el jugador y recuerda su última posición conocida. Cuando deja de verlo durante el tiempo establecido, pasa a una etapa de búsqueda/alerta.

Valores base identificados: velocidad de patrulla 30, velocidad de alerta 40 y velocidad de persecución 55; distancia de llegada 5; tiempo de pérdida de visión 1,5 s; tiempo máximo de atasco durante investigación 2 s. Estos valores son parámetros exportados y pueden variar por instancia.

El sistema incluye control de atasco para evitar investigaciones indefinidas: si el guardia no logra desplazarse lo suficiente en el intervalo previsto, retorna a patrulla.

7. Linternas, obstáculos, visión y captura

El cono visible comunica al jugador la dirección de observación. La verificación de línea de visión contempla obstáculos y contribuye a que las paredes funcionen como cobertura. Las reacciones de varios guardias pueden encadenarse indirectamente cuando el jugador corre y genera ruido cerca de otros enemigos.

El cono de linterna se construye mediante un Polygon2D y múltiples raycasts, con distancia de luz de referencia 180, ángulo de 55 grados y 25 rayos. Los raycasts permiten adaptar visualmente el cono a obstáculos del escenario.

La comprobación de visión considera distancia, ángulo y una consulta física que verifica la línea de visión y el posible bloqueo por paredes. El sistema representa la orientación del guardia mediante animaciones de ocho direcciones y offsets de la linterna según la dirección.

Durante la persecución, el contacto físico con el jugador puede producir derrota. En el tutorial, la captura genera una señal específica para reposicionar al protagonista y reiniciar la patrulla, sin finalizar la experiencia de aprendizaje.

La alarma aumenta la velocidad de los guardias mediante un multiplicador configurable de referencia 1,3, combinado con el multiplicador de dificultad.

8. Interacciones, llaves e inventario

El botón de interacción permite actuar sobre objetos cercanos. Las llaves pueden estar sueltas en el escenario o ser transportadas por guardias. El diseño aconseja aproximarse por detrás en sigilo, pero los métodos de entrega de llave revisados no imponen por sí mismos ese requisito.

Las llaves del escenario son Area2D con id_llave exportado y método quitar_llave(). Al recogerse, entregan su identificador, se registran en SaveManager.llaves_recogidas y se eliminan de la escena mediante queue_free(). Al cargar un checkpoint, las llaves ya recogidas no reaparecen.

El inventario del jugador se representa mediante Array[String]. agregar_llave(), tiene_llave() y consumir_llave() permiten almacenar, consultar y consumir identificadores. La señal llaves_actualizadas comunica los cambios al HUD.

Los guardias también implementan quitar_llave(): si llevan un identificador, pueden entregarlo al jugador y vaciar su campo id_llave. La interacción del jugador es polimórfica por disponibilidad del método; el script de guardia no verifica por sí mismo la orientación trasera ni el estado de sigilo en quitar_llave().

El HUD muestra los identificadores de las llaves en líneas separadas; no se documenta como un sistema de iconos gráficos, porque eso no está presente en el código revisado.

9. Puertas y progresión dentro de los niveles

Las puertas heredan de StaticBody2D. Cada instancia define id_llave_necesaria, region_cerrada y region_abierta. intentar_abrir(player) comprueba si ya está abierta, permite apertura libre cuando el identificador está vacío y, si requiere llave, llama a player.consumir_llave(id_llave_necesaria).

Cuando se abre, cambia la región visible de Sprite2D y desactiva la colisión mediante CollisionShape2D.set_deferred("disabled", true). Se reproduce door_open.wav; si falta la llave, se reproduce door_locked.wav.

Las puertas abiertas se registran por nombre de nodo en SaveManager.puertas_abiertas. En _ready() recuperan su estado cuando existe checkpoint válido. Es importante mantener identificadores de nodos consistentes y evitar colisiones de nombres entre puertas lógicamente distintas.

10. Tesoros, shader, alarma y escape

El apagón se mantiene durante la huida. Al robar el objetivo, el juego pasa de la infiltración a una carrera contra el reloj, con modificaciones en el comportamiento de los guardias.

El tesoro es StaticBody2D con script @tool y textura exportada, lo que permite sustituir su representación desde el Inspector y actualizar Sprite2D en el editor. robar() evita duplicaciones, marca el tesoro como robado, avisa a GameManager y elimina el objeto.

El shader canvas_item de brillo utiliza texture(TEXTURE, UV), una onda sinusoide basada en TIME y parámetros uniform intensidad_brillo (0 a 0,5; predeterminado 0,2) y velocidad_brillo (0,1 a 5; predeterminado 2). Modifica los canales RGB y conserva el alfa original.

GameManager.registrar_robo_tesoro() activa la alarma y el contador. El tiempo base es 120 segundos, modificado según dificultad. La fase de escape termina al llegar al punto de salida con el tesoro o al producirse una condición de derrota.

11. Condiciones de victoria, derrota y salida

En los niveles principales, una zona de salida Area2D detecta body_entered, verifica que el cuerpo se llame Player y comprueba GameManager.tiene_tesoro. Si se cumple, llama a GameManager.completar_nivel().

completar_nivel() detiene el temporizador, registra la progresión, elimina el checkpoint y emite nivel_completado. El HUD muestra «ROBO COMPLETADO» y habilita CONTINUAR. Tras el tercer nivel, la navegación conduce a final_juego.tscn.

La derrota puede originarse por captura de guardias o agotamiento del tiempo. El HUD muestra «HAS SIDO ATRAPADO», ofrece REINTENTAR y, si corresponde, la carga del checkpoint del nivel. La salida del tutorial utiliza una señal distinta, tutorial_completado, sin desbloquear niveles de campaña.

12. Escenarios y progresión de dificultad

Los tres escenarios reutilizan la misma lógica de jugador, guardias, llaves, puertas y tesoro. El diseño busca una dificultad creciente mediante distribución espacial, rutas de patrulla, posibilidades de ocultamiento y presión durante el escape; no se atribuyen valores concretos diferentes por nivel sin respaldo de las escenas.

El videojuego se denomina The Hidden Thief. El protagonista es un ladrón que aprovecha una serie de apagones para realizar robos cada vez más ambiciosos. El juego se estructura inicialmente en tres niveles:

Nivel 1 - Museo: el objetivo es robar una estatuilla antigua.

Nivel 2 - Mansión privada: el objetivo es robar una joya de gran valor.

Nivel 3 - Banco: el objetivo es robar un gran diamante.

Al completar un nivel se mostrará una pantalla de felicitación por haber obtenido el tesoro y luego una pantalla que presenta el siguiente objetivo. Al completar el Banco se mostrará una pantalla final que comunica que el jugador se ha convertido en un ladrón legendario, con opción de volver al menú principal.

El menú principal ofrece Fácil, Normal y Difícil. SaveManager define multiplicadores de velocidad de guardias: 0,8; 1,0; y 1,2 respectivamente. El tiempo de alarma se calcula con multiplicadores 1,25; 1,0; y 0,75 sobre 120 s, resultando 150, 120 y 90 s.

Los parámetros de velocidad, audición, luz, tiempos de búsqueda y patrulla son configurables por script o instancia. El balance final se considera parte de la versión 1.0; esta documentación registra valores identificados en el código, no resultados de una batería formal de pruebas.

13. Persistencia, checkpoint y recuperación

SaveManager utiliza user://partida.json con FileAccess y JSON.stringify. Guarda nivel desbloqueado, dificultad, volumen de música y efectos, flags de partida y checkpoint, nivel guardado, posición del jugador, inventario de llaves, llaves recogidas, puertas abiertas, tesoro robado, alarma y tiempo restante.

El checkpoint también registra datos de guardias por nombre: posición, llave transportada, estado de IA, posición sospechosa, última posición conocida del jugador, fase de alerta y temporizadores de búsqueda. La restauración del nivel aplica esos valores y llama a actualizar_estado_restaurado() para recomponer el comportamiento.

El guardado manual se solicita desde el HUD mediante SaveManager.guardar_checkpoint(player.global_position, GameManager.nivel_actual, player.llaves). La apertura de puertas y la recogida de llaves modifican datos en memoria; su persistencia en disco depende de una operación de guardado.

Para reiniciar un nivel sin restaurar el checkpoint se usa SaveManager.ignorar_checkpoint. La opción CARGAR del HUD restablece explícitamente ese indicador a false; el menú principal invoca GameManager.cargar_nivel() sin ese restablecimiento explícito. Se registra como consideración de mantenimiento, no como defecto confirmado en la versión distribuida.

14. Interfaz, HUD, pausa y resultados

El HUD utiliza CanvasLayer. Se conecta a alarma_activada, tiempo_actualizado, tiempo_agotado, jugador_derrotado y nivel_completado de GameManager, además de llaves_actualizadas del jugador.

El temporizador permanece oculto antes de la alarma y presenta los segundos restantes mediante ceil(). El inventario se representa mediante texto, incluyendo la cantidad cero o los identificadores disponibles.

Un panel compartido presenta PAUSA, DERROTA y VICTORIA mediante un enum. Durante la pausa, puede guardarse manualmente si permitir_guardar es verdadero. Al reanudar, se restablece get_tree().paused = false. El panel también ofrece volver al menú y cargar un checkpoint cuando existe.

En victoria, CONTINUAR carga la presentación del siguiente nivel; tras superar el Banco, carga la escena final.

15. Menú principal, opciones y configuración

El menú ofrece Nueva partida, Continuar, Opciones, Tutorial y Salir. Continuar se deshabilita cuando SaveManager.hay_guardado_manual es falso. Nueva partida abre una pantalla de dificultad y, al confirmar, borra los datos previos, almacena la dificultad, marca la partida iniciada y comienza la presentación del Museo.

Las opciones utilizan dos HSlider para música y efectos. Los valores 0–100 se convierten a 0–1, se aplican inmediatamente mediante AudioManager y se guardan automáticamente en JSON.

El botón Tutorial abre una escena independiente. Salir invoca get_tree().quit().

16. Tutorial independiente

El tutorial se organiza mediante un controlador Node2D y un RichTextLabel que muestra instrucciones y mensajes coloreados. Empieza pausado, con una pantalla de consejos; COMENZAR la oculta y reanuda el juego.

La progresión exige: utilizar ocho direcciones; correr; desplazarse en sigilo; recoger una llave; abrir la primera puerta; robar una llave al guardia; abrir una segunda puerta; robar el tesoro; y regresar a la salida.

El progreso de movimiento se guarda en un diccionario de ocho banderas. Las otras etapas se validan mediante entradas y estados de los objetos: desaparición de llave/tesoro, puertas abiertas e identificador de llave vacío en el guardia.

Ante captura, el jugador reaparece en un Marker2D y el guardia vuelve a patrulla. La zona de salida del tutorial emite tutorial_completado solo cuando GameManager.tiene_tesoro es verdadero. La escena presenta una felicitación y permite regresar al menú sin afectar el avance de campaña.

17. Presentación de niveles y cierre de campaña

presentacion_nivel.gd reutiliza una única escena para los tres títulos: «NIVEL 1 - EL MUSEO», «NIVEL 2 - LA MANSIÓN» y «NIVEL 3 - EL BANCO». Reproduce una animación de ventana y mueve al ladrón hacia ella con Tween.

La secuencia temporal incluye desplazamiento de 3 s, pausa de 0,5 s, aparición del título en 1,5 s, permanencia de 2 s y desvanecimiento de 1,5 s. Después se carga el nivel actual mediante GameManager.cargar_nivel().

La pantalla final reproduce música específica al abrirse. El botón de regreso detiene esa música, elimina la pausa, reinicia el estado temporal de GameManager y vuelve al menú principal. El texto exacto de felicitación depende de la configuración de la escena y no de este script.

18. Recursos visuales, sonido y música

El estilo gráfico utiliza pixel art con personajes animados en ocho direcciones, entornos interiores, objetos interactivos y representación visible de linternas. La presentación visual y la iluminación de cada escena se configuran en el editor de Godot; el código revisado acredita la lógica de detección y animación de las linternas, pero no todas las propiedades visuales de las escenas.

AudioManager crea reproductores AudioStreamPlayer para música y efectos, asociados a los buses Music y SFX. La configuración persistida se transforma de volumen lineal a decibelios con linear_to_db() y se aplica a los buses.

La música normal utiliza res://audio/pink_panther.mp3; la alarma, res://audio/alarm.wav; la pantalla final, res://audio/musica_final.mp3. Los efectos de puerta y otros eventos se disparan desde los scripts correspondientes.

Consideración de publicación: verificar derechos y licencias de cada pista antes de distribuir públicamente el ejecutable. Un único reproductor SFX puede reemplazar un efecto anterior si se inicia otro sonido sobre el mismo reproductor.

19. Comunicación entre sistemas y señales

GameManager emite tiempo_actualizado(float), alarma_activada, tiempo_agotado, tesoro_robado, jugador_derrotado y nivel_completado. Player emite llaves_actualizadas(int). El guardia del tutorial emite jugador_atrapado_tutorial; la zona de salida del tutorial emite tutorial_completado.

El uso de señales permite que el HUD y el tutorial reaccionen a eventos sin depender de un sondeo continuo de todos los estados globales. El uso de métodos como quitar_llave(), robar() e intentar_abrir() permite compartir la lógica de interacción entre objetos de distintas clases.

20. Validación, riesgos y mantenimiento

Riesgos técnicos abordados durante el desarrollo: coherencia entre cono de luz y detección, obstáculos que bloquean visión, balance de velocidades y ruido, navegación de múltiples guardias, persistencia consistente y alcance acotado de la campaña. La confirmación de funcionamiento no equivale a una certificación formal de pruebas automatizadas.

El desarrollador confirma que la versión 1.0 está terminada y que los tres niveles, el tutorial y el sistema de guardado funcionan. Se ha informado una exportación ejecutable de Windows x86_64 con ventana 1152 × 648, relación 16:9 y paquete PCK integrado.

Consideraciones de mantenimiento derivadas de la revisión: conservar identificadores únicos de llaves y puertas; verificar el restablecimiento de ignorar_checkpoint en todas las rutas de carga; validar que la configuración de bucle del WAV de alarma sea la deseada; y revisar las licencias de audio para distribución.

Estas consideraciones no se presentan como errores reproducidos. Un parche 1.0.1 podría concentrarse en correcciones; una versión 1.1 podría incorporar mejoras de usabilidad o nuevas características. Se recomienda registrar cambios y pruebas por versión.

21. Referencias y antecedentes de diseño

Material de la cátedra: “Documento Técnico de Diseño” (Emanuel Segretín). Se tomó como referencia su separación entre GDD y TDD y sus secciones sugeridas: descripción técnica, objetivos, herramientas, riesgos, guías de estilo, sistemas, recursos externos, versionado, plataformas y detalles de entidades/sistemas.

Documento de ejemplo: “Technical Design Document”, Emma Smith, Kyle Culbert & Zubin Vandrewalla. Se utilizó como referencia de organización para características, niveles, interacción, colisiones, IA, herramientas y plataforma; no se copiaron sus decisiones de implementación.